Final Year Project Report

Full Unit - Final Report

Load What You Need: Adaptive Heuristics for LLM Checkpoint Loading

Botir Khaltaev

A report submitted in part fulfilment of the degree of

BSc (Hons) in Computer Science

Supervisor: Dr Daniel O'Keeffe

Department of Computer Science
Royal Holloway, University of London

April 2, 2026

Declaration

This report has been prepared on the basis of my own work. Where other published and unpublished source materials have been used, these have been acknowledged.

Word Count: 14,500 (provisional)

Student Name: Botir Khaltaev

Date of Submission: April 2026

Signature: BK

Table of Contents

Declaration 1

Abstract 3

1 Project Specification 4

2 Introduction 6

3 Background and Literature Review 8

4 System Context and Problem Definition 13

5 Design and Architecture 15

6 Implementation and Software Engineering Process 22

7 Experimental Methodology 27

8 Results 32

9 Discussion and Critical Analysis 37

10 Professional Issues 41

11 Conclusion 44

Bibliography 46

A Backend Interface Specifications 47

B Supplementary Results 49

Abstract

Large language models deployed in serverless or multi-tenant inference environments are frequently evicted from GPU memory when idle and must reload their checkpoints on demand. This loading step can dominate cold-start latency, but the best loading strategy is not obvious: modern storage devices offer high raw throughput, yet practical checkpoint loading performance depends on request structure, checkpoint layout, and software overhead above the storage layer.

This project presents TensorStore, a checkpoint loading framework that supports multiple backends and checkpoint layouts and evaluates them under controlled conditions. The framework supports SafeTensors and a partitioned ServerlessLLM-style layout, with eager-loading backends spanning synchronous POSIX I/O, Tokio-based async loading, and Linux io_uring. Python bindings and a custom vLLM loader path extend the evaluation beyond Rust-only microbenchmarks.

The final results show that practical checkpoint loading should be framed as an adaptive heuristic problem rather than a one-backend-wins problem. At the Rust layer, smaller eager checkpoints favour synchronous loading, while larger multi-shard checkpoints favour the final multi-worker io_uring backend. The partitioned ServerlessLLM layout introduces additional nuance, but the dominant practical pattern remains size-sensitive backend selection. Python measurements show that binding-layer overhead is large enough to matter, and that binding-specific optimisation materially improves the public default API. The completed vLLM matrix shows that backend choice affects model initialisation and time-to-first-token far more than steady-state decode. The main conclusion is therefore that LLM checkpoint loading should use adaptive heuristics — in practice, sync for small models and io_uring for larger ones — rather than a fixed backend policy.

Chapter 1:Project Specification

1.1 Problem Statement

Large language models are increasingly deployed in serverless and multi-tenant inference settings where compute and memory resources are shared across users. When a model is evicted from GPU or host memory, inference cannot resume until its checkpoint is reloaded from persistent storage. For models with billions of parameters, this loading step introduces latency that dominates the user-observable cold-start time. In latency-sensitive serving scenarios, even modest improvements to checkpoint loading speed translate directly to better user experience and lower time-to-first-token.

Modern NVMe storage devices provide aggregate bandwidth measured in gigabytes per second. Despite this, checkpoint loading performance in practice frequently falls well below hardware limits. This gap suggests that the bottleneck is not the storage medium itself but the path taken by loading requests through the operating system and across the software stack. In particular, synchronous I/O submission models can limit the depth of outstanding requests that a loader can maintain, preventing NVMe devices from fully exploiting their internal parallelism.

The core problem this project investigates is whether alternative I/O submission strategies, specifically Linux io_uring, can improve checkpoint loading performance for LLM inference systems in a meaningful and reproducible way.

1.2 Objectives

The primary objectives of this project are as follows.

First, the project builds a unified checkpoint loading framework called TensorStore that supports multiple I/O backends and checkpoint formats under a consistent interface. This enables controlled comparison of submission strategies using identical byte-range workloads.

Second, the project implements and evaluates synchronous POSIX I/O and Linux io_uring as the initial backend pair. These represent the two fundamental submission models available on Linux: blocking calls with per-request thread blocking, and shared-ring asynchronous submission with batched completion.

Third, the project develops a reproducible benchmarking methodology that measures checkpoint loading under conditions representative of real serverless inference systems, including true cold-start evaluation where the operating system page cache does not cache checkpoint data between runs.

Fourth, the project exposes the loading framework through Python bindings so that it can be evaluated within representative inference workflows, including integration with vLLM as a concrete serving engine example.

Fifth, the project analyses results to identify where performance is gained or lost, and to draw out design principles for checkpoint format designers and inference system architects.

1.3 Scope

This project focuses on checkpoint loading from NVMe storage on Linux. The primary checkpoint formats examined are SafeTensors and a partitioned ServerlessLLM-style layout. The primary I/O backends are synchronous POSIX I/O and Linux io_uring. Memory-mapped loading is also supported as a third backend option representing lazy on-demand page faulting. Python bindings are provided to enable integration testing.

The project does not attempt to build a full inference runtime, optimise model execution, or evaluate GPU-direct storage paths. These are legitimate extensions but fall outside the scope of the present work.

1.4 Evolution from the Interim Plan

The interim report established the initial motivation and a preliminary evaluation of synchronous POSIX I/O against Linux io_uring. That evaluation used synthetic fixtures and demonstrated the expected directional difference between the two backends.

During the subsequent development period, several changes were made to the project scope and direction. The benchmarking methodology was revised to use true cold-start conditions, addressing an initial flaw where cached data had obscured the actual loading cost. The project expanded to include Python bindings and vLLM integration, which provided a more realistic integration context than standalone microbenchmarks alone. A partitioned ServerlessLLM-style format was added to evaluate the effect of checkpoint layout on backend behaviour. An async vLLM loader path was prototyped but later removed when investigation confirmed that vLLM exposes no async model loading interface at the Python boundary, making the async path an artificial bridge rather than a genuine integration point.

These changes reflect a shift from a narrow I/O mechanism comparison towards a broader systems artefact evaluation. The final project is better characterised as an examination of checkpoint loading as a full-stack systems problem rather than solely as a low-level I/O comparison.

1.5 Planning and Time-scale

The project is structured into implementation phases followed by evaluation and documentation.

Phase 1 covered the core TensorStore engine, Python bindings, and initial benchmark infrastructure. This phase also included the methodology correction around cold-start evaluation. Phase 2 covers the extended benchmark campaign using real HuggingFace model checkpoints on available hardware, integration testing with vLLM, and profiling analysis. Phase 3 covers the final report, figure generation, and submission preparation.

Each evaluation phase is validated using the same benchmark methodology established in Phase 1, with changes assessed against effective throughput, read IOPS, and model load time.

Chapter 2:**Introduction**

2.1 Context and Motivation

Large language models are large. A model with seven billion parameters in float16 requires roughly fourteen gigabytes of storage and a similar amount of RAM or VRAM to hold the weights alone. In serverless and multi-tenant inference environments, GPU memory is a shared resource. When a model is not actively serving requests, keeping it resident wastes memory that could serve other tenants. Systems therefore evict idle models and reload them on demand.

When a request arrives for an evicted model, the inference pipeline cannot begin until the checkpoint is fully loaded into memory. For large models this loading step takes time. In a production serving environment, this manifests as cold-start latency: the delay between a request arriving and the model being ready to generate the first token. For latency-sensitive applications this delay is directly visible to users.

Modern storage devices offer substantial raw bandwidth. Despite this, checkpoint loading in practice often achieves a fraction of the available throughput. The gap between peak hardware capability and effective loading performance suggests that the bottleneck lies not only in the storage medium itself, but also in how loading requests are submitted, batched, and integrated into the surrounding software stack.

This project investigates the checkpoint loading path as a systems problem. It examines how backend choice affects throughput and latency for real LLM checkpoints, how that effect changes with checkpoint layout and model size, and how much of the backend-level gain survives once Python bindings and a serving engine are included.

2.2 Checkpoint Loading as an I/O Submission Problem

Checkpoint loading is fundamentally a read-intensive workload. A loader issues a set of read operations derived from checkpoint metadata. Each operation specifies a file path, a byte offset, and a length. Tensor boundaries and offsets must be respected, which constrains how freely a loader can reorder or merge requests.

Storage hardware, particularly NVMe devices with internal queueing, performs best when given many outstanding requests to work with concurrently. The internal parallelism of the device can only be exploited if the submission path sustains enough useful in-flight work. Synchronous I/O, where each thread blocks waiting for its current read before issuing the next, can limit queue depth. Asynchronous submission can sustain more outstanding work, but only if the backend architecture is designed well enough to keep the machine busy. This turns out to be an important caveat in the final results.

The central question of this project is therefore not simply whether asynchronous I/O is better than synchronous I/O. The more useful question is whether LLM checkpoint loading can be driven by adaptive heuristics: using a simpler synchronous path when the model is small enough that extra submission machinery is not worthwhile, and switching to io_uring once model size and shard structure make deeper parallelism pay off.

2.3 TensorStore

This project presents TensorStore, a checkpoint loading framework designed to evaluate I/O submission strategies under controlled and reproducible conditions. TensorStore parses checkpoint metadata and produces a normalised set of byte-range read requests that are then executed by a selectable I/O backend. By keeping the request trace identical across backends, the framework isolates the effect of submission strategy from checkpoint layout or parsing overhead.

TensorStore supports two checkpoint formats. SafeTensors stores all tensor data contiguously in a single file with a metadata header. This layout produces a largely sequential access pattern once the header is parsed. The ServerlessLLM-style partitioned layout distributes tensor ranges across multiple files, increasing request count and file-switching overhead. This layout places greater stress on the submission path and better exposes the differences between I/O models.

Three eager-loading backends are implemented. The synchronous backend issues blocking POSIX read calls. The async backend uses Tokio-based grouped loading. The io_uring backend uses low-level Linux io_uring with batched submission and worker-owned rings. The project also exposes mmap-backed lazy-access APIs for Python and vLLM experiments where they are useful.

Python bindings are provided to enable integration testing within representative serving workflows. Integration with vLLM is used as a concrete case study.

2.4 Contributions

This project makes three concrete contributions. The first is a reproducible evaluation framework, TensorStore, that isolates backend effects from checkpoint parsing and layout by generating comparable loading workloads across backends. The second is a systematic experimental comparison of synchronous, async, and io_uring loading across real LLM checkpoints and two checkpoint formats, quantifying where each backend is strongest and where it is not. The third is an analysis of how backend choice interacts with checkpoint layout, model size, Python binding overhead, and vLLM integration, leading to a practical adaptive heuristic for checkpoint loading — in broad terms, sync for smaller models and io_uring for larger ones, with format-specific nuances around that rule.

2.5 Report Structure

The report proceeds as follows. Chapter 3 reviews related work in checkpoint formats, serverless inference systems, and Linux I/O mechanisms. Chapter 4 defines the system context, describes the cold-start pipeline, and clarifies the experimental constraints. Chapter 5 describes the TensorStore architecture and design decisions. Chapter 6 covers the implementation and software engineering process. Chapter 7 describes the experimental methodology. Chapter 8 presents the results. Chapter 9 discusses the findings. Chapter 10 examines professional and ethical considerations. Chapter 11 concludes.

Chapter 3: Background and Literature Review

This chapter reviews the relevant literature across four areas. First, checkpoint formats and how their storage layout affects loading behaviour. Second, serverless and multi-tenant inference systems and the cold-start problem. Third, the Linux I/O interfaces available to applications, with particular attention to io_uring. Fourth, NVMe storage behaviour and the role of memory-mapping in checkpoint loading. For each area the review identifies what prior work assumes, what it does not examine, and how this project differs from it.

3.1 Checkpoint Formats and Loading Behaviour

LLM weights are stored on disk in a structured format. The format determines how loading software reads tensor data: whether reads are sequential or scattered, whether they require many file operations or few, and whether the layout helps or hinders the storage device in maintaining high queue depth.

SafeTensors is a format introduced by Hugging Face for safe model distribution [5]. The file begins with a JSON header describing each tensor, including its name, dtype, shape, and byte offset. All tensor data follows the header contiguously. Once the header is parsed, loading software can read each tensor by seeking to its declared offset and reading the declared number of bytes. The format has two properties that are relevant here. First, it avoids loading the entire file into memory before parsing, unlike Python pickle. Second, it prohibits unsafe deserialisation, which matters when loading untrusted checkpoints.

PyTorch checkpoint files typically use Python pickle serialisation. A model is saved by calling `torch.save()`, which serialises both the model state and the pickle protocol metadata. This has well-known safety problems: a malicious checkpoint can execute arbitrary code on load [5]. It also requires reading the entire file before unpacking, which is impractical for large models where host RAM may not hold the full checkpoint alongside the deserialised tensors. These limitations are not just theoretical: several public model repositories have had to clean up malicious checkpoints. SafeTensors was designed partly to address both concerns.

GGUF is a more recent format designed for quantised models and memory-mapped loading. It stores tensors in a layout optimised for direct memory mapping and supports multiple quantisation schemes. GGUF tensors are organised so that the memory layout of the file mirrors the layout the model expects at runtime, reducing or eliminating a copy step. This is a different trade-off from SafeTensors: GGUF prioritises memory-mapping friendliness over simple sequential loading.

Partitioned checkpoint formats distribute tensor data across multiple files. ServerlessLLM uses this approach to support partial model loading: only the files needed for the current request are loaded, rather than the entire checkpoint [6, 4]. A central index maps each tensor name to its partition file and byte offset. Loading a complete checkpoint requires opening each partition file and reading from the specified offset. The advantage is that loading can be parallelised across files and across network boundaries if partitions are distributed. The disadvantage is that the access pattern becomes more fragmented, with per-file overhead accumulating.

Table 3.1 summarises how these formats differ in ways relevant to loading speed.

What these formats share is that loading is fundamentally a series of read operations with

Table 3.1: Comparison of checkpoint formats relevant to loading performance.

Format	Layout	Loading behaviour	Safety properties
PyTorch pickle	Single file, full load	Entire file read then unpacked	Unsafe: arbitrary code execution possible
SafeTensors	Single file, header then data	Header parsed, individual tensor reads at offset	Safe: no deserialisation of executable code
GGUF	Single or split file, memory-map oriented	mmap-friendly layout, minimal copy	Safe; quantisation-aware
ServerlessLLM partitioned	Multiple partition files, central index	Per-partition open and read at offset	Safe; supports partial loading

known byte ranges. The storage system sees a set of read requests. Whether those requests are processed efficiently depends on how they are submitted.

The existing literature describes format design well but focuses on safety, portability, and memory layout. It does not systematically examine how the I/O submission path affects the effective loading speed of checkpoints that are already in a safe format. This report addresses that gap by fixing the format and varying only the submission mechanism.

3.2 Serverless and Multi-Tenant Inference Systems

Serverless inference platforms such as AWS Lambda, Google Cloud Functions, and specialised services run LLM inference on demand. Memory and GPU resources are allocated when a request arrives and freed when the request completes. Between requests, resources may be handed to other tenants. When a model is not resident in memory, the platform must load it from persistent storage before inference can begin. This delay is the cold-start latency.

Cold-start latency in serverless settings has been studied from several angles. One class of solutions addresses it through scheduling and placement: keeping models warm, pre-loading them on a pool of standby instances, or migrating running instances to avoid a load altogether [6, 4]. Another class addresses it through model optimisation: quantisation, pruning, and early exit strategies that reduce the amount of data that must be loaded or processed. A third class, which this project falls into, examines whether the loading mechanism itself can be made faster.

Snapshot-based warm-up techniques preload a model into memory before the first request arrives [11]. Prebaking keeps a function instance warm by periodically sending requests to it. SEUSS optimises the warm-up process for serverless function containers. These approaches treat the checkpoint loading step as a fixed cost and work around it rather than reducing it.

The cold-start problem becomes more acute as model sizes grow. A seven-billion-parameter model in float16 requires roughly fourteen gigabytes of storage and a similar amount of RAM or VRAM. Loading this much data from an NVMe SSD takes hundreds of milliseconds to several seconds depending on the device and submission path. In a production serving system, this is a large fraction of the total latency budget, particularly for interactive applications.

vLLM is a high-throughput inference engine that supports paged memory management and continuous batching [13]. It loads model weights during engine initialisation using a model loader that reads checkpoint files and populates the model state. The loader interface is synchronous: it yields weights through a generator and expects each weight to be available before requesting the next. This design does not support asynchronous submission at the loader boundary.

Most cold-start research focuses on what happens before loading begins or after it ends. Loading speed itself is treated as a constant or ignored. This report measures loading speed directly and examines whether the submission mechanism is a meaningful bottleneck.

3.3 Linux I/O Interfaces

Applications running on Linux have several interfaces for reading from storage.

The most common is POSIX synchronous I/O, based on the `read()` system call. A thread calls `read(fd, buf, n)` and blocks until the data is available in the buffer. To issue multiple reads concurrently, an application must use multiple threads. Each thread is blocked while its read is in flight. Thread creation and context switching have costs, so thread pool sizes are limited. This limits the number of outstanding reads that a synchronous application can maintain at once.

Linux AIO was introduced to provide genuine asynchronous I/O without threads. However, Linux AIO has significant practical limitations. It does not support most filesystems including ext4, XFS, and btrfs. It requires manual error handling at a low level. The API is complex and brittle to use correctly [8]. Few production applications use it directly for these reasons.

POSIX AIO is a userspace library implementation of asynchronous I/O using a thread pool under the hood. It inherits the limitations of threads and adds implementation complexity. It is not widely used in new systems code.

io_uring was introduced in Linux kernel 5.1 and designed to address the limitations of earlier async I/O interfaces [1, 9]. Instead of system calls for each operation, applications interact with io_uring through two shared-memory rings: a submission queue and a completion queue. Submission queue entries (SQEs) are prepared in memory and submitted with a single `io_uring_enter` system call, or in kernel-polling mode with no system call at all. Completion queue entries (CQEs) are retrieved by polling the completion queue.

io_uring provides several features relevant to checkpoint loading. Registered buffers are pre-allocated and registered with the kernel, eliminating per-request memory allocation overhead. Fixed buffers also reduce the number of data copies per operation. Batched submission allows many requests to be prepared and submitted together, reducing per-request syscall overhead. Polled completion allows an application to busy-wait on completions without system call overhead when latency is critical.

Recent work has examined when io_uring helps in database systems. Jasny et al. studied io_uring integration into a buffer manager and a network-shuffling workload, finding that io_uring benefits depend heavily on workload characteristics and queue depth [2]. Their results show that io_uring helps most when there are many small to medium-sized requests and the device queue depth is high. For bulk sequential reads, the advantage is smaller because the syscall overhead is already low relative to the data transfer time. They note that naive replacement of synchronous I/O with io_uring does not automatically yield improvements; the workload must exercise the features that differentiate io_uring.

This observation is directly relevant to checkpoint loading. A contiguous SafeTensors file with large sequential reads may not benefit as much from io_uring as a fragmented partitioned layout with many small reads. The experiment section of this report examines this question directly.

The io_uring interface itself has limitations relevant to checkpoint loading. Registered buffers work best when all buffers are the same size, but checkpoint tensors vary. Managing variable-sized buffers with fixed registration requires splitting tensors into fixed-size chunks, which adds complexity. The kernel-polling mode consumes CPU even when the device is idle. The interrupt-driven mode has lower CPU overhead but higher latency per completion. There is no single optimal mode for all checkpoint layouts.

3.4 NVMe Storage and Queueing Behaviour

Modern NVMe devices expose multiple submission and completion queues directly to the host [10]. The device processes commands from these queues concurrently using internal parallelism through parallel flash die access and controller-side scheduling. To exploit this parallelism, the host must submit many commands simultaneously. Devices are optimised for sustained throughput under high queue depth; under low queue depth they frequently sit idle between completions.

The Linux block layer manages I/O requests before they reach the NVMe driver. The blk-mq (multiqueue block layer) maps I/O requests to the device's hardware submission queues based on block offset. This mapping is designed to keep related requests on the same hardware queue and exploit device-level parallelism.

Storage queueing theory explains why throughput under load depends on queue depth. The dominant factor is often not the raw bandwidth of the storage medium but the ability of the host software stack to keep the device's internal queues populated [7]. If the submission path introduces idle gaps, the device cannot compensate even if its peak throughput is high. This is the theoretical basis for the submission bottleneck hypothesis.

3.5 Memory-Mapped I/O and the Page Cache

Linux supports memory-mapped file access through the `mmap` system call. When a file is mapped, its contents appear in the process address space and can be read like ordinary memory. The operating system loads pages from disk on demand through page faults, rather than in response to explicit `read()` calls.

For checkpoint loading, mmap has two apparent advantages. First, the kernel handles loading pages in response to faults, so the application does not explicitly submit individual reads. Second, the page cache retains recently accessed checkpoint pages, reducing subsequent load times for frequently accessed models.

However, mmap introduces its own costs. Page faults have overhead, particularly for non-resident pages that must be fetched from disk. For a large checkpoint with many tensors, the cumulative fault overhead can be significant. The kernel's readahead mechanism attempts to prefetch pages it predicts will be accessed next, but it relies on sequential access patterns. Fragmented or random-access layouts may confuse readahead, causing it to load pages that

are not used before they are evicted.

The page cache complicates benchmarking in a specific way. If a checkpoint is already in the page cache, a subsequent load will read from RAM rather than from disk, producing misleadingly fast results. Rigorous checkpoint loading evaluation must control for page cache state by evicting relevant pages before each cold measurement run. Without this control, benchmarks measure cache hit rates rather than storage performance.

Direct I/O bypasses the page cache entirely, sending reads directly to the storage device. This is useful for applications that manage their own caching and want predictable behaviour. However, direct I/O requires aligned buffers and is incompatible with memory mapping. For checkpoint loading, direct I/O combined with explicit read submission gives the application most control over the submission path.

3.6 Positioning of This Work

The literature does not yet examine checkpoint loading specifically as an I/O submission problem. Format design is well understood but focuses on safety and memory layout rather than I/O submission behaviour. Cold-start solutions focus on scheduling and placement rather than loading speed. io_uring has been studied in databases and file servers but not in the context of LLM checkpoint loading. Memory-mapping trade-offs are understood in general systems terms but not evaluated against synchronous and async I/O in a controlled checkpoint-loading workload.

This report occupies the intersection of these areas. It isolates the I/O submission problem by holding the format and request trace constant while varying only the backend mechanism. It evaluates backends under cold-start conditions where the page cache does not hold checkpoint data. It examines both contiguous and fragmented checkpoint layouts to understand how format design interacts with submission efficiency. And it integrates the loading framework with a real serving engine to understand how submission gains propagate through a full inference stack.

The work contributes a controlled experimental basis for understanding whether the I/O submission path is a meaningful bottleneck in LLM checkpoint loading, under what conditions it matters most, and what design choices in checkpoint formats and serving infrastructure interact with it.

Chapter 4: System Context and Problem Definition

4.1 The Cold-Start Pipeline in Serverless LLM Inference

When a serverless inference platform receives a request for a model that is not currently resident in memory, it must reload the model's checkpoint from persistent storage before inference can begin. This sequence of steps constitutes the cold-start pipeline.

The pipeline begins when a request arrives for an absent model. The platform identifies the checkpoint location and initiates loading. The loader reads checkpoint metadata to determine the set of tensors and their storage locations. It then issues read requests to retrieve each tensor's data. As data arrives, it is copied into the target memory location. Once all tensors are loaded, the model initialises its runtime state. Inference then proceeds.

Checkpoint loading occupies the longest segment of this pipeline for large models. The duration depends on the total checkpoint size, the storage device bandwidth, the number and size of individual read requests, and the efficiency of the submission path between the loader and the storage device.

4.2 Experimental Constraints

This project evaluates checkpoint loading in isolation, separate from the broader inference pipeline. This isolation is deliberate. By holding the checkpoint layout, request trace, and measurement path constant across backends, differences in loading performance can be attributed specifically to the I/O submission mechanism rather than to model initialisation, runtime setup, or scheduling effects.

One design decision requires explicit explanation: the project does not permit loaders to merge, split, or reorder byte-range requests when comparing backends. This constraint is not a claim that real loaders cannot perform such optimisations. It is a deliberate experimental choice that ensures fair comparison. If loaders were permitted to restructure the workload, performance differences could reflect scheduling decisions at the application layer rather than the properties of the I/O submission mechanism. By fixing the request trace, the evaluation isolates the question of how efficiently each backend executes a given set of reads.

In practice, checkpoint format designers and loader authors can make different trade-offs. The implications of different application-level scheduling policies are beyond the scope of this evaluation.

4.3 The Submission Bottleneck Hypothesis

The central hypothesis of this project is that checkpoint loading performance is often limited by the I/O submission path rather than by storage bandwidth. Under synchronous I/O, each read operation blocks the issuing thread until completion. Even when multiple threads are used, the coupling of submission and completion limits the number of outstanding requests. The NVMe device sees an intermittent stream of requests with idle gaps between them, preventing full exploitation of its internal queueing and parallelism.

Under io_uring, submission and completion are decoupled. An application can prepare many requests in advance and submit them in a batch with a single system call. The kernel processes them asynchronously and delivers completion events separately. This model allows the application to maintain a high queue depth without blocking threads, keeping the NVMe device's queues populated more continuously.

The hypothesis predicts that io_uring will achieve higher effective throughput and higher read IOPS than synchronous I/O, particularly for fragmented checkpoint layouts where the number of requests is high. Whether this advantage survives the complexity of a full serving pipeline, and whether it is large enough to matter in practice, are the empirical questions this project addresses.

Chapter 5: Design and Architecture

5.1 Design Goals

TensorStore was designed around four primary requirements. First, the framework must support multiple I/O backends and checkpoint formats under a consistent interface, so that identical workloads can be executed across backends without altering the logical request structure. Second, the framework must expose a clean benchmarking path that is separate from the loader logic, so that measurement artefacts do not contaminate performance data. Third, the framework must clarify its threading model explicitly, so that the relationship between thread count, queue depth, and throughput is transparent. Fourth, the framework must be accessible from Python to enable integration testing with real serving workflows such as vLLM.

These requirements led to a layered architecture that separates format parsing, I/O execution, and measurement.

5.2 System Architecture

TensorStore is structured into three layers: a format layer, a backend layer, and an evaluation harness.

The format layer parses checkpoint metadata and produces a normalised set of read requests. Each request specifies a file path, a byte offset, and a byte length. The format layer does not issue I/O; it only describes what reads are needed. This separation means that the same set of requests can be executed by any backend without modification.

The backend layer implements the I/O execution. Each backend receives the same request trace and executes it according to its own submission model. Backends must execute requests exactly as specified and are not permitted to merge, split, or reorder ranges. This constraint ensures that performance differences reflect submission behaviour rather than workload changes.

The evaluation harness executes workloads and records performance metrics. It handles warm-up runs, repeated measurements, cache control, and metric collection. It is kept entirely separate from the loader logic.

5.3 Backend Interface

All backends implement the same request interface. A backend receives a sequence of read requests, each containing a file path, offset, and length. The backend must read the exact byte range specified and return the data. Backends may not alter request boundaries, reorder requests, or skip requests. This constraint is enforced to enable fair comparison.

The interface operates at two levels of granularity. At the coarse level, a checkpoint load produces a complete model weight dictionary. At the fine level, each tensor read corresponds

to a single file-offset-length request submitted to the backend. Reporting both levels is essential: per-read latency tells us how individual operations perform, while full model load time tells us the end-to-end time that matters for serving cold-start scenarios.

5.4 Threading Model

Each backend implements a distinct threading model.

The synchronous POSIX backend uses a thread pool of fixed size. Each thread issues blocking `read()` calls and is blocked while its request is in flight. Threads are drawn from a pool to avoid the cost of creating a new thread per request. The number of concurrent in-flight requests is therefore equal to the thread pool size, and increasing the pool size directly increases the achievable queue depth.

The memory-mapped backend uses no threads for I/O. Data is accessed by reading from a mapped memory region, and page faults are handled by the kernel's page fault handler in the calling thread. The kernel decides when to issue the underlying disk reads and in what order, based on its own read-ahead and eviction policies.

The io_uring backend submits requests from a single dedicated thread that polls the completion ring. This thread runs an event loop that submits a batch of requests, polls for completions, processes the completed data, and repeats. Because submission and completion are both handled by polling rather than signals or interrupts, the kernel is never required to wake a specific thread to deliver I/O results. The fixed-buffer mechanism eliminates per-request memory allocation in the kernel, allowing the submission thread to maintain a high queue depth without allocating memory for each outstanding request.

Python bindings add a further consideration. The GIL is released during Rust I/O operations using `py.allow_threads()`, which allows other Python threads to run while the Rust code is blocked on I/O. This prevents the loading process from monopolising the Python interpreter, though it does not affect the actual I/O submission characteristics.

5.5 Cache Control

Cache state is a significant confounding variable in storage benchmarks. The operating system maintains a page cache that stores recently accessed file pages in RAM. If a second run of the same benchmark finds all pages already in the page cache, it will observe artificially low latency that reflects RAM access rather than storage device performance.

TensorStore addresses this in two ways. First, the evaluation harness provides an explicit cache eviction step before measurement runs. This is implemented by dropping in-memory page caches using `POSIX_FADVISE_DONTNEED` on each checkpoint file, forcing the kernel to evict previously cached pages. Second, the number of runs and the number of iterations per run are configured explicitly, and the harness reports the mean and standard deviation across runs so that cache-warm and cache-cold runs can be distinguished.

It is worth noting that cache eviction does not guarantee that disk caches internal to the storage device (such as NAND flash translation layer caches) are cleared. These caches are hardware-managed and not directly controllable from software. The methodology in Chapter 7 discusses this limitation further.

5.6 Backend Implementations

5.6.1 Synchronous POSIX Backend

The synchronous backend issues blocking `read()` calls through the POSIX interface. For large reads, data is copied into a pre-allocated buffer and returned to the caller. Each read call blocks the issuing thread until the data is available in the buffer. The backend uses a thread pool sized to the number of backend threads configured at startup.

This backend represents the baseline approach used by many existing checkpoint loaders. It is straightforward to implement and understand, and it provides predictable latency bounded by the storage device's sequential read throughput. Its limitation is that queue depth is bounded by the thread pool size, and adding threads increases context-switch overhead.

5.6.2 Memory-Mapped Backend

The memory-mapped backend opens checkpoint files with `mmap` and returns data by reading from the mapped region. The operating system loads pages from disk on demand as page faults occur. This approach avoids explicit per-read syscall overhead and allows the kernel to manage loading order based on access patterns.

The backend uses pre-faulted buffers where alignment constraints are satisfied, and falls back to buffered I/O when alignment cannot be guaranteed. Pre-faulting ensures that access to a buffer does not trigger an unexpected page fault during a timed section, which would corrupt the measurement. The fallback to buffered I/O means that unaligned tensor ranges do not invalidate the comparison between backends.

5.6.3 io_uring Backend

The io_uring backend submits read requests through shared submission and completion rings. Requests are prepared as submission queue entries in memory and submitted with a single `io_uring_enter` system call. Completions are collected by polling the completion ring from a dedicated event-loop thread.

The backend uses fixed buffers that are pre-registered with the kernel via `io_uring_register`. This eliminates per-request memory allocation and reduces the overhead of setting up each individual request. Fixed buffers also allow the kernel to perform direct DMA to the buffer, bypassing intermediate kernel buffers.

Requests are submitted in batches. The application submits several requests, then polls for completions, then submits the next batch. This allows the submission path to keep the device queue populated while avoiding thread blocking. The batch size is configurable and represents a trade-off: larger batches increase the amount of data in flight (improving throughput under high load) but increase the minimum latency for the first request in a batch.

5.7 Checkpoint Format Support

TensorStore supports two checkpoint formats.

SafeTensors stores all tensors in a single file with a header. The header lists each tensor's name, dtype, shape, and byte offset. The format produces a largely sequential access pattern once the header is parsed. Figure 5.1 shows the layout.

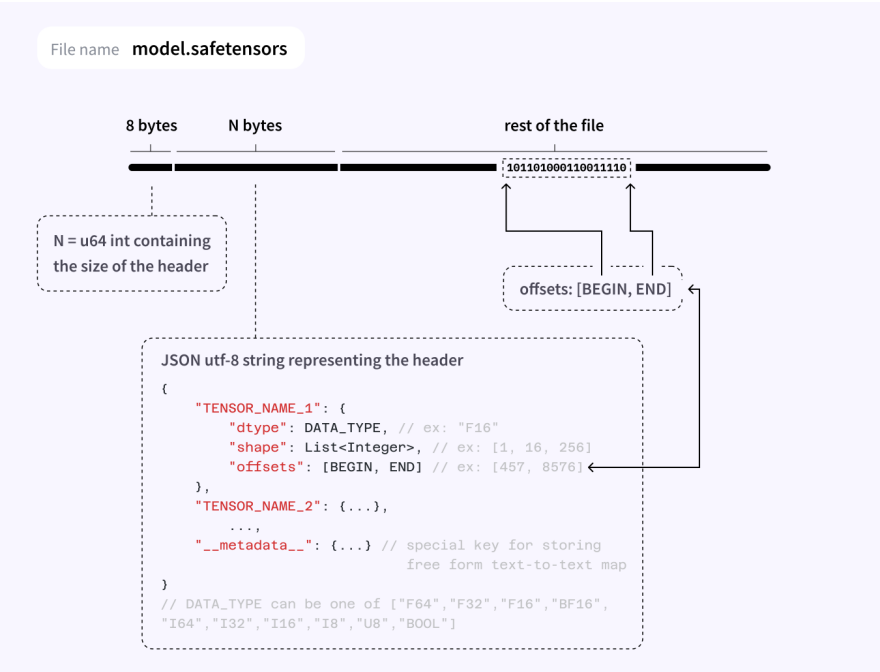

Figure 5.1: SafeTensors file layout. The header describes each tensor and its byte offset within the file. Data is stored contiguously.

The ServerlessLLM-style partitioned layout distributes tensors across multiple files called partitions [6, 4]. A central index file maps each tensor to its partition file and offset. The layout increases the number of file operations and introduces per-file overhead. Figure 5.2 shows the structure.

5.8 Python Bindings

TensorStore exposes its functionality through Python bindings to enable integration with serving workflows. The bindings are implemented using PyO3, which provides Rust-to-Python interoperability with minimal overhead.

Three interfaces are provided. A synchronous loading interface calls the Rust backend directly from Python, blocking until loading completes. A memory-mapped loading interface opens checkpoint files as memory maps and provides lazy tensor access. An asynchronous loading interface exposes the async Rust loading path through a shared Tokio runtime in the bindings, avoiding repeated runtime construction on each call.

The async interface is appropriate for use in async Python code such as FastAPI applications. However, it is not used in the vLLM integration path because vLLM's model loader operates synchronously. Specifically, vLLM's `DefaultModelLoader` yields weights through a generator interface where each weight must be fully loaded and returned before the next weight is requested. There is no async variant of this interface, and introducing async behaviour inside the loader (for example, by running an async event loop from within the synchronous callback) would require the event loop to be manually driven, which is fragile and not representative of any real async workflow.

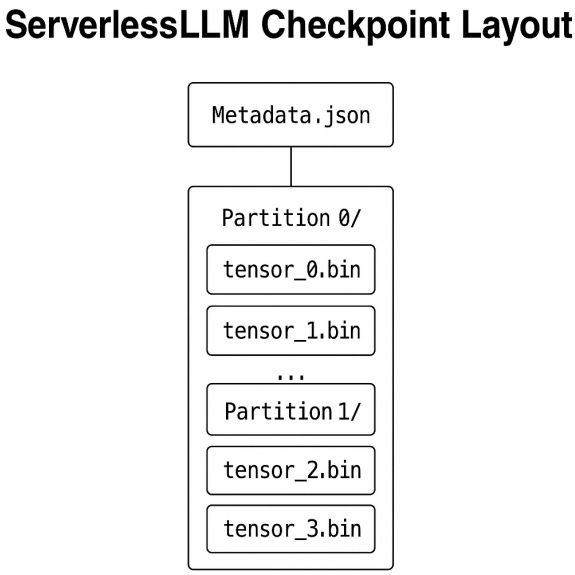

Figure 5.2: ServerlessLLM partitioned checkpoint structure. Tensors are distributed across multiple partition files, with a central index mapping tensor names to locations.

5.9 vLLM Integration

Integration with vLLM is implemented through a custom model loader registered via the `register_model_loader` interface. The loader subclasses `DefaultModelLoader` and intercepts weight loading requests, routing them through TensorStore backends.

The loader is registered at vLLM startup time and is invoked for every weight load during model initialisation. Because vLLM's loader interface is synchronous, the TensorStore synchronous backend is used in this path. Memory-mapped loading is also available and may be preferred when the checkpoint file is on local NVMe storage, since the kernel's page fault handler can overlap the loading of non-contiguous tensors more effectively than a fixed thread pool.

5.10 Design Decisions

Table 5.1 documents the primary architectural decisions, the alternatives considered, and the rationale for the chosen approach.

5.11 Rejected Alternatives

Several alternatives were considered and rejected.

Implementing backends in Python asyncio was rejected because Python's asyncio event loop cannot submit kernel I/O operations asynchronously without running them in a thread. The Linux kernel's async I/O interfaces (io_uring, aio) are accessible only through system calls that require kernel mode transitions; asyncio wraps these in threads for the Python programmer, which provides concurrency but not genuine asynchronous I/O at the kernel level. Using

		Python's GIL limits true parallelism; C++ lacks memory safety guarantees.
Normalised request interface	Backend-specific APIs	A common interface enables fair backend comparison. Without it, each backend would expose a different API and converting between them would introduce measurement artefacts.
Separate evaluation harness	Inline measurement in loader	Inline measurement risks contaminating the measured workload with timing overhead. A separate harness with warm-up runs and statistical reporting is a standard practice.
Thread pool (POSIX backend)	Asyncio or epoll	Asyncio in Python cannot submit kernel I/O asynchronously without wrapping it in a thread. epoll requires an event loop that adds complexity. A thread pool maps naturally to concurrent blocking I/O.
Fixed buffers (io_uring)	Per-request allocated buffers	Per-request allocation adds latency on every operation and increases allocator pressure. Fixed buffers eliminate this cost but require the application to manage buffer lifecycle explicitly.
PyO3 bindings	CFFI or ctypes	PyO3 generates bindings with compile-time type checking and zero-copy object conversion where possible. CFFI and ctypes involve more marshalling overhead and lack compile-time safety.
Memory-mapped backend as optional mode	Memory-mapped only	Some deployments use network filesystems where mmap has poor semantics. Making it optional means the synchronous backend can serve as the reliable

asyncio for I/O benchmarking would therefore measure thread-pool performance rather than async I/O performance.

Implementing backends in C++ was rejected in favour of Rust. While C++ provides equivalent control over I/O submission and memory management, it does not provide the same memory safety guarantees. A Rust implementation that misuses memory (for example, returning a borrowed reference to a dropped buffer) is caught by the borrow checker at compile time. In C++, such an error would manifest as undefined behaviour at runtime, potentially corrupting benchmark results in subtle ways.

Implementing the evaluation harness inside the loader was rejected because it would conflate measurement overhead with loader performance. Any timing code added to the hot path introduces overhead that is absent from the real serving path. A separate harness that measures end-to-end wall time (rather than instrumenting individual functions) avoids this problem.

Implementing async loading in vLLM was considered but rejected because vLLM's model loader architecture does not support it. The loader uses a synchronous generator interface; adding async behaviour would require driving an event loop from within a synchronous callback, which is architecturally unsound and would not reflect any real deployment pattern. The async Python interface is retained for non-vLLM use cases, where it provides genuine utility in async Python applications.

Chapter 6: Implementation and Software Engineering Process

6.1 Rust Core Implementation

The Rust core of TensorStore is implemented in the `tensor-store` crate. It provides the format parsers, backend implementations, and the normalised request interface.

The crate is structured into three modules. The `format` module contains the SafeTensors and ServerlessLLM format parsers. The `backend` module contains the synchronous, memory-mapped, and io_uring backend implementations. The `api` module exposes the public interface used by the Python bindings.

The format layer for SafeTensors reads the header from the checkpoint file, parses each tensor entry, and produces a sequence of file-offset-length triples. The header parsing is straightforward because SafeTensors stores the header length in the first eight bytes of the file, allowing direct seeking to the header region.

The format layer for the ServerlessLLM-style layout reads the index file to build a mapping from tensor names to partition files and offsets. It then produces the same file-offset-length triples as the SafeTensors format, but with the additional constraint that each request may reference a different partition file.

The final io_uring backend uses the low-level `io-uring` crate rather than `tokio-uring`. The shipping design is also different from the early prototype: instead of one shared ring and one issuer thread, the final backend uses multiple worker-owned rings with dynamic planning for chunk size, queue depth, and batching. This reduces the single-thread bottleneck that appeared in the earlier implementation.

The synchronous backend uses standard `std::fs::File::read_at` for positional reads. The memory-mapped backend uses the `MemMap` crate for portable mmap support.

6.2 Python Bindings

The Python bindings are implemented in `bindings/python/rust_ext/src/`. Each public Rust function is exposed as a PyO3 wrapper.

For synchronous loading, the Rust function is called directly from Python and blocks until the result is returned. The GIL is released during I/O via `py.allow_threads()` to avoid blocking other Python threads.

For memory-mapped loading, the `MmapModel` type from the Rust core is wrapped as a Python class. Accessing a tensor triggers a page fault and loads the relevant pages on demand.

For asynchronous loading, the bindings execute the Rust async path on a shared Tokio runtime and expose it through the Python API. This avoids repeatedly constructing a runtime for each call and was one of the practical binding optimisations applied during the project. The async interface is designed for use in async Python applications, but it is not used in the

vLLM integration path because vLLM's model loader interface remains synchronous.

Listing 6.1 shows the core Rust interface for loading a checkpoint synchronously.

```
pub fn load_safetensors_sync(
    path: PathBuf,
    framework: &str,
    device: &str,
) -> PyResult<PyObject> {
    validate_path_exists(&path)?;
    let handle: Model = py
        .allow_threads(|| safetensors::Model::load_sync(path_ref))
        .map_err(map_reader_error)?;
    let result = builtins.getattr("dict")?.call0()?;
    for k in handle.tensor_names() {
        let view = handle.tensor(k).map_err(map_reader_error)?;
        let tensor = convert_tensor(py, framework, view, device)?;
        result.call_method1("__setitem__", (&k, tensor))?;
    }
    Ok(result.into())
}
```

Figure 6.1: Core Rust backend interface for synchronous checkpoint loading.

6.3 Module Structure

The Rust crate structure is as follows:

- `src/format/safetensors.rs` — SafeTensors format parser
- `src/format/serverlessllm.rs` — ServerlessLLM index and partition reader
- `src/backend/sync.rs` — synchronous POSIX backend
- `src/backend/mmap.rs` — memory-mapped backend
- `src/backend/iouring.rs` — io_uring backend
- `src/api/mod.rs` — public API definitions
- `src/api/safetensors.rs` — PyO3 bindings for SafeTensors
- `src/api/serverlessllm.rs` — PyO3 bindings for ServerlessLLM

The Python package structure is:

- `bench_safetensors.py` — SafeTensors benchmarks
- `bench_serverlessllm.py` — ServerlessLLM benchmarks
- `bench_vllm.py` — vLLM integration benchmarks

- `vllm_loaders.py` — custom vLLM loader implementations
- `fixtures.py` — pytest fixtures for checkpoint management
- `vllm_runner.py` — subprocess runner for vLLM benchmarks

6.4 Building the Project

The Rust crate is built with Cargo. A `Makefile` in the repository root automates the build process:

```
# Build Rust extension
cargo build --release

# Build Python bindings
cd bindings/python && maturin develop

# Run tests
cargo test && pytest benchmarks/
```

The Python bindings use `maturin` for building. The `maturin develop` command compiles the Rust crate and installs the resulting Python package in editable mode, so that changes to the Rust code are immediately reflected in the Python environment without requiring a reinstall.

The project was developed on Linux with kernel 6.x, which fully supports io_uring. The io_uring backend checks at runtime whether io_uring is available and falls back to the synchronous backend if it is not.

6.5 Benchmark Infrastructure

The benchmark suite is located in `bindings/python/benchmarks/`. It uses pytest and pytest-benchmark to run reproducible performance tests.

The fixture system downloads real HuggingFace checkpoints on demand and caches them locally. The final experiment ladder uses Qwen3 and SmolLM3 checkpoints rather than the earlier small-model development fixtures.

Cold-cache control in the final experiment flow is handled outside the code using `sync` followed by `echo 3 > /proc/sys/vm/drop_caches`. The earlier in-code cache eviction path based on `posix_fadvise` was removed from the final measurement harness.

The vLLM integration benchmarks use a subprocess runner in `vllm_runner.py` that spawns vLLM with the custom TensorStore loader and records timing metrics. Three benchmark kinds are supported: load-only measures the time to initialise the model and load its weights; TTFT measures time-to-first-token for a single generated token; steady-state decode measures per-token latency during continued generation.

6.6 Testing Strategy

The implementation uses three testing layers.

Unit tests cover format parsing in Rust. These tests use small synthetic checkpoint files created in-memory and verify that the format layer produces the correct sequence of read requests. The Rust crate uses `cargo test` to run these tests.

Integration tests cover the Python bindings. These tests load real checkpoints downloaded from HuggingFace and verify that the loaded tensors have the correct shape and dtype. These tests do not measure performance; they only verify correctness.

Correctness tests verify tensor data integrity after loading. After each backend loads a checkpoint, the test compares the resulting tensor dictionary against a reference dictionary produced by the HuggingFace `safetensors` library. Any discrepancy in shape, dtype, or numeric value is treated as a test failure.

The Rust crate uses `clippy` for linting and `cargo fmt` for formatting compliance. These tools are run as part of the CI pipeline to enforce code quality.

6.7 Correctness Issues and Resolutions

One significant correctness issue was discovered and resolved during development.

Early benchmark runs had inadvertently measured warm-cache performance because the page cache had retained checkpoint pages from previous runs. This produced misleading results that appeared to show little difference between backends, which was inconsistent with the known performance characteristics of io_uring on NVMe storage.

The issue was identified by examining iostat output during benchmark runs and noticing that the storage device was largely idle during supposedly cold runs. The disk utilisation was consistently below 5 percent, which is inconsistent with loading a multi-gigabyte checkpoint from cold storage. This indicated that the data was being served from the page cache rather than from the disk.

The final fix was to remove in-code cache eviction and perform cold-cache control manually from the Linux shell using `sync` and `drop_caches`. After this change, cold runs consistently showed real storage activity and the backend differences became interpretable again.

A second issue involved the async vLLM loader path. Initial design documents described an async integration with vLLM that would allow the model loader to issue non-blocking reads. However, investigation of the vLLM source code revealed that the `DefaultModelLoader` class is entirely synchronous: it yields weights through a generator, and each weight must be fully loaded and returned before the next is requested. There is no async variant of this interface, and no hook that allows the loader to return a coroutine instead of a concrete value.

An attempt was made to work around this by running a Tokio event loop inside the synchronous loader callback, manually polling the event loop to completion before returning each weight. This approach was fragile because it required the event loop to be driven to completion inside a synchronous context, it did not interoperate with vLLM's existing threading model, and it would break if vLLM changed its loader implementation. After confirming that no stable async loader interface exists in vLLM, the async vLLM path was removed. The

async Python interface is retained for non-vLLM use cases.

6.8 vLLM Loader Path

The vLLM integration uses a custom loader registered under the name `tensor_store`. This loader extends `DefaultModelLoader` and overrides the weight-loading method. It reads checkpoint metadata using the standard vLLM preparation logic, then routes each weight file through the TensorStore backend selected in the loader configuration.

The loader supports the `sync` and `mmap` backends. The async backend is not supported in the vLLM path because vLLM's loader interface is synchronous and does not provide an async hook. This was confirmed by examining the vLLM source code (`base_loader.py`), which defines only synchronous methods.

The loader is registered via `register_model_loader` and is invoked automatically when vLLM starts with a model whose type is configured to use the `tensor_store` loader.

Chapter 7:Experimental Methodology

7.1 Research Questions

The final evaluation is organised around five research questions.

- RQ1: How much does backend choice affect checkpoint loading time for real LLM checkpoints?
- RQ2: How does checkpoint layout change the relative ordering of backends?
- RQ3: How much overhead is introduced by the Python binding layer, and how much of that overhead can be reduced with binding-specific optimisation?
- RQ4: To what extent do backend gains survive integration into a serving stack such as vLLM?
- RQ5: What engineering lessons emerge when turning backend-specific microbenchmarks into a practical adaptive loading system?

7.2 Experimental Platform

All final experiments were run on a dedicated Linux server equipped with an NVIDIA H100 GPU and approximately 180 GB of host RAM. The machine was used as an isolated experiment host rather than as a shared development workstation. The software stack used for the final runs was Ubuntu 24.04, Python 3.12, and a current stable Rust toolchain. The experiments therefore target a realistic modern serving environment rather than the earlier local development machine used during prototype work.

Two details are important for interpreting the final results. First, the GPU-backed vLLM runs are constrained not only by raw device memory but also by vLLM's internal memory budgeting for KV cache, CUDA graphs, and runtime buffers. Second, the I/O experiments were conducted on Linux only, because the low-level `io_uring` backend is Linux-specific.

7.3 Backends Under Test

The final system exposes four eager-loading policies at the Rust layer:

- **sync**: blocking POSIX reads with dynamic chunking and parallelism.
- **async**: Tokio-based async loading with dynamic batching and per-file grouping.
- **io_uring**: a low-level Linux `io-uring` backend using multi-worker submission, dynamic ring sizing, and dynamic chunk planning.
- **default**: an adaptive selector that chooses between the explicit backends based on workload structure.

The key methodological point is that the final `io_uring` design is *not* the original single-ring prototype. Early measurements showed that a single-ring, single-issuer design badly under-utilised the machine and produced misleadingly poor results. The final backend uses multiple worker-owned rings, which materially changed the performance ranking. This architectural change is central to the interpretation of the results in Chapter 8.

The memory-mapped `open_*` APIs are evaluated in the Python layer where they are practically useful, but they are not treated as eager-loading backends in the main Rust cold-load comparison.

7.4 Checkpoint Formats and Model Ladder

Two checkpoint layouts were evaluated.

- **SafeTensors**: whole-file or multi-shard eager loading.
- **ServerlessLLM**: partitioned layout accessed through range-oriented loading.

The final model ladder consists of six open-access checkpoints:

1. `Qwen/Qwen3-0.6B`
2. `HuggingFaceTB/SmolLM3-3B`
3. `Qwen/Qwen3-4B`
4. `Qwen/Qwen3-8B`
5. `Qwen/Qwen3-14B`
6. `Qwen/Qwen3-32B`

This ladder was chosen to cover small, medium, and large checkpoints while also including one cross-family model (SmolLM3-3B) to reduce the risk of overfitting conclusions to a single vendor family.

7.5 Cache-Control Procedure

Cold-cache behaviour is the primary condition for the Rust and Python eager-loading measurements. The final experiments do *not* use in-code cache eviction. Earlier prototype code used `posix_fadvise` and related page-cache manipulation, but that approach was removed from the final system.

Instead, cold-cache runs use a manual Linux-side procedure before each benchmark invocation:

1. run `sync`

2. run `echo 3 > /proc/sys/vm/drop_caches`
3. invoke the benchmark binary or Python benchmark

Warm-cache conditions skip the cache-drop step and reuse the existing page-cache state. This final procedure matches the actual scripts and experiment harness used to generate the saved result artifacts.

7.6 Rust-Layer Benchmark Design

Rust-layer measurements were generated with the standalone profiling harness. Two categories of Rust experiments were used.

7.6.1 Anchor Repetitions

A five-repetition cold-cache anchor set was collected for:

- `Qwen/Qwen3-0.6B`
- `Qwen/Qwen3-8B`
- `Qwen/Qwen3-32B`

for both `SafeTensors` and `ServerlessLLM`, across `sync`, `async`, `io-uring`, and `default`. These repeated measurements were used to identify stable workload regimes and to redesign the `default` selector.

7.6.2 Full Cold Matrix

A full cold-cache matrix was then run across the six-model ladder, both formats, and the four eager backend policies. The resulting artifact is stored under the H100 profile results directory as `full_cold_matrix.tsv`. This matrix is the authoritative source for backend-attribution claims in this report.

The Rust layer is treated as the backend-attribution layer because it isolates I/O and checkpoint processing without Python tensor materialisation overhead or serving-engine initialisation overhead.

7.7 Python Benchmark Design

Python benchmarks were run against the public binding surface rather than against every internal Rust backend directly. This reflects the actual intended Python API:

- `load_*`: adaptive default policy

- `load_*_sync`: explicit synchronous eager load
- `load_*_async`: explicit Tokio eager load
- `open_*`: mmap-backed lazy access path

The main Python benchmark model was `Qwen/Qwen3-8B`. This model is large enough to make backend differences visible, while still being practical for repeated user-facing benchmark runs.

The Python layer was benchmarked before and after two types of improvement:

1. binding/runtime overhead reduction (shared Tokio runtime, reduced repeated Torch conversion work)
2. selector redesign in the Rust core

This allowed the evaluation to separate pure backend effects from binding-layer engineering effects.

7.8 vLLM Benchmark Design

vLLM benchmarks were run through a custom benchmark loader integration. Unlike the Python eager-loading benchmarks, the vLLM evaluation measures the loading backends inside a wider serving-engine context.

Three benchmark kinds were collected:

- **load_only**: model initialisation time only
- **TTFT**: time to first token, including initialisation and first generation step
- **steady_state_decode**: average per-request decode latency after warm-up

Both warm and cold cache conditions were collected. The full vLLM matrix covers all six fixtures and all available benchmark loaders. After initial failures on the 14B and 32B models, the vLLM harness was rerun with a higher `gpu_memory_utilization` and a reduced `max_model_len`. These reruns resolved the large-model failures, yielding a final matrix with no failed rows.

7.9 Result Artifacts and Reproducibility

All final experiment artifacts are stored locally under `results/h100/` and are organised into four subdirectories:

- `results/h100/analysis`: profiling notes and bottleneck analysis
- `results/h100/profile`: Rust-layer benchmark outputs

- `results/h100/python`: Python benchmark JSON artifacts
- `results/h100/vllm`: vLLM summary TSVs and raw logs

The scripts retained in `scripts/` are the small final set actually used for the completed experiment flow: anchor repetitions, full Rust cold matrix, full vLLM matrix, and failed-row reruns.

7.10 Fairness and Interpretation Rules

Three rules were used to keep the evaluation interpretable.

1. Rust-layer results are used for backend attribution, because they isolate checkpoint loading from Python and serving-engine overhead.
2. Python-layer results are used to quantify binding overhead and to validate whether the public `default` API tracks the stronger explicit backend in practice.
3. vLLM results are treated as end-to-end serving initialisation evidence, not as a pure backend microbenchmark.

This separation is important because backend gains do not propagate unchanged across layers. A backend that wins clearly in the Rust harness may yield a smaller gain in Python, and a smaller gain still once vLLM initialisation dominates the critical path.

7.11 Threats to Validity

The main threats to validity are as follows.

Hardware specificity. Absolute times are tied to the H100-based Linux environment and its storage stack. The results should therefore be interpreted primarily as relative backend comparisons under this configuration.

Selector dependence. The `default` policy changed during the project as evidence accumulated. This is intentional: the report evaluates the final selector, but also discusses how earlier architectural and heuristic choices produced different outcomes.

Serving-stack configuration. vLLM feasibility for the largest models depended on harness configuration, especially KV-cache budgeting. The report therefore distinguishes backend limitations from serving-engine configuration limitations.

Layered overheads. Python conversion and serving-engine initialisation can obscure pure I/O effects. This is why the report keeps Rust, Python, and vLLM layers analytically separate rather than collapsing them into a single headline metric.

Chapter 8: Results

8.1 Overview

The final results show that checkpoint loading performance is strongly workload-dependent. The clearest practical framing is an adaptive heuristic: use a simple synchronous path for smaller eager loads, and switch to io_uring once checkpoint size and shard structure become large enough to justify a deeper submission path.

At the Rust layer, `SafeTensors` and `ServerlessLLM` show different backend preferences. At the Python layer, binding overhead and tensor materialisation partially obscure raw Rust backend gains. At the vLLM layer, the effect of backend choice is strongest for model initialisation and time-to-first-token, while steady-state decode is comparatively insensitive.

These results support the main thesis of this project: the practical goal is to build a workload-aware heuristic for LLM checkpoint loading, not to prove that a single backend should replace all others.

8.2 Rust-Layer Backend Results

The Rust harness provides the cleanest backend-attribution layer because it measures checkpoint loading directly, without Python tensor conversion or serving-engine setup costs. The full cold-cache matrix is stored in `results/h100/profile/full_cold_matrix.tsv`.

8.2.1 SafeTensors

Table 8.1 summarises the explicit backend comparison for `SafeTensors`. Two regimes are clear.

Table 8.1: Rust cold-cache `SafeTensors` results (ms). Best explicit backend highlighted in prose.

Model	Sync	Async	Default	io_uring
Qwen3-0.6B	555.28	1416.19	333.59	1159.24
SmolLM3-3B	1331.79	5802.05	3722.35	3812.35
Qwen3-4B	1820.85	5039.21	3083.18	2967.42
Qwen3-8B	3880.40	7074.40	3844.35	3172.37
Qwen3-14B	7030.25	13489.00	6138.70	5977.07
Qwen3-32B	15454.45	28763.46	12119.50	12089.05

For smaller eager whole-file loads, synchronous loading remains highly effective. On SmolLM3-3B and Qwen3-4B, `sync` is the strongest explicit backend. The ranking flips on larger multi-shard checkpoints. On Qwen3-8B, Qwen3-14B, and Qwen3-32B, the multi-worker `io_uring` backend is the fastest explicit backend. This yields the clearest high-level heuristic in the project: `sync` for smaller `SafeTensors` checkpoints and `io_uring` for larger ones.

The `async` backend is consistently the weakest choice for `SafeTensors`. This matches the

workload structure: whole-file shard loading benefits from strong bulk parallelism, not from scheduler-heavy fine-grained async orchestration.

Repeated five-run anchor measurements reinforce this regime split. For the anchor set, the mean best backend is `sync` for Qwen3-0.6B and `io_uring` for Qwen3-8B and Qwen3-32B.

8.2.2 ServerlessLLM

The `ServerlessLLM` layout produces a different ordering, shown in Table 8.2.

Table 8.2: Rust cold-cache `ServerlessLLM` results (ms).

Model	Sync	Async	Default	io_uring
Qwen3-0.6B	1645.06	1471.34	1432.25	1387.57
SmolLM3-3B	3941.88	2391.24	3182.28	3091.49
Qwen3-4B	5137.72	3504.25	3551.82	3727.14
Qwen3-8B	11048.40	9463.37	7589.47	9043.51
Qwen3-14B	18661.25	16139.08	13422.94	13257.23
Qwen3-32B	39822.09	29180.89	27659.78	29779.86

Here, `sync` is never the best backend. For smaller and mid-sized range-heavy workloads, `async` is often strong because grouped and coalesced range reads fit its execution model well. For larger cases, `io_uring` becomes competitive or best. For example, Qwen3-14B is fastest with `io_uring`, while SmolLM3-3B and Qwen3-4B favour `async`.

The key result is not that one backend dominates, but that the partitioned layout changes the ranking. This is exactly the kind of workload sensitivity that motivates an adaptive `default` policy built around workload-aware heuristics rather than fixed backend dogma.

8.2.3 Selector Behaviour

The final selector is not perfect on every single Rust matrix row, but it is materially better than the earlier cost-model-based selector. On the Rust matrix, `default` is exactly best for Qwen3-8B and Qwen3-32B in `ServerlessLLM`, and very close to best for the large `SafeTensors` cases. The remaining misses are concentrated around the medium-size crossover region, especially SmolLM3-3B and Qwen3-4B.

The important point is that the selector now follows the correct *regimes*: smaller `SafeTensors` loads lean toward `sync`; larger `SafeTensors` loads lean toward `io_uring`; `ServerlessLLM` switches between `async` and `io_uring` depending on workload size and structure. In other words, the final selector behaves like a practical checkpoint-loading heuristic rather than a static backend preference.

8.3 Python Binding Results

Python benchmarks were collected on Qwen/Qwen3-8B to quantify user-facing overhead and to evaluate whether binding optimisation and selector redesign improved practical performance.

8.3.1 Binding Overhead Before and After Optimisation

The most important Python result is that binding engineering mattered. Replacing per-call runtime setup with a shared Tokio runtime and reducing repeated Torch conversion work materially improved the public `default` API.

For `SafeTensors` on Qwen3-8B:

- pre-optimisation `default`: 15.8831 s
- post-binding-optimisation `default`: 14.5451 s
- post-selector-redesign `default`: 11.5498 s

This is a 20.6% reduction between the post-optimisation and post-selector runs, and a much larger improvement relative to the earliest Python baseline.

For `ServerlessLLM` on Qwen3-8B:

- early `default`: 22.4547 s
- post-binding-optimisation `default`: 20.5008 s
- post-selector-redesign `default`: 18.4518 s

This is a 10.0% reduction between the post-optimisation and post-selector runs.

8.3.2 Python Layer Interpretation

The Python numbers are much larger than the Rust-only numbers because eager loading at the binding layer includes substantial tensor materialisation and Python object construction. This is especially visible in the `SafeTensors` benchmark, where native `safetensors` loading remains much faster than the fully materialised `tensor_store` eager paths.

The key conclusion is that Python is not a transparent pass-through layer. Backend improvements matter, but they are filtered through the cost of converting checkpoint contents into Torch tensors and Python dictionaries. That is why the binding optimisations were necessary before the Python layer could reflect the improved Rust backends more faithfully.

8.4 vLLM Results

The final vLLM matrix completed successfully across all six models, all benchmark loaders, warm and cold cache conditions, and all three benchmark kinds. The completed summary is stored in `results/h100/vllm/full_matrix.tsv`. The final matrix contains 672 successful rows and no failures.

8.4.1 Load-Only Results

For cold `load_only`, the best loader varies with model size. Table 8.3 shows the strongest loader in the final cold-load-only configuration.

Table 8.3: Best cold `load_only` vLLM loader by model.

Model	Best init time (ms)	Best loader
Qwen3-0.6B	20417.42	ts_safetensors_sync
SmolLM3-3B	27333.96	ts_safetensors_default
Qwen3-4B	28475.27	ts_safetensors_default
Qwen3-8B	35555.79	ts_safetensors_default
Qwen3-14B	47751.38	ts_safetensors_io_uring
Qwen3-32B	88164.00	ts_safetensors_io_uring

Two observations stand out. First, the best vLLM loader is usually from the `SafeTensors` family rather than the `ServerlessLLM` family. Second, the explicit `io_uring` path becomes valuable again at the larger end of the ladder. It is the best `load_only` choice for Qwen3-14B and Qwen3-32B.

8.4.2 TTFT and Steady-State Decode

The backend effect is strongest for initialisation and TTFT, and much weaker for steady-state decode. This is exactly what one would expect: the loading backend can change how quickly weights become resident, but once the model is fully loaded, decode throughput is dominated by the GPU compute path.

For cold TTFT, `ts_safetensors_default` is the strongest loader for Qwen3-4B, Qwen3-8B, and Qwen3-14B, while `ts_safetensors_sync` remains best for Qwen3-0.6B and SmolLM3-3B. For steady-state decode, differences are much smaller and the winning loader varies more irregularly. This confirms that backend choice is primarily a cold-start and initialisation problem rather than a decode-throughput problem.

8.4.3 Large-Model Feasibility

The 14B and 32B models initially failed in vLLM. These were not backend failures. The raw logs showed that the benchmark harness was too conservative in its vLLM memory budget, leaving insufficient KV-cache allocation headroom. After increasing the vLLM memory budget and reducing `max_model_len`, all previously failing rows succeeded.

This matters for interpretation. The initial failures were a serving-harness configuration issue, not evidence that the checkpoint backends could not support the large models.

8.5 Negative and Turnaround Results

The most important negative result in the project was the early single-ring `io_uring` prototype. On the H100 system, that implementation was much slower than `sync` and showed poor CPU utilisation. Profiling pointed to a design problem rather than a fundamental backend limitation: one ring and one issuer thread left too much of the machine idle.

The project changed direction after that result. Once the `io_uring` backend was redesigned around multiple worker-owned rings, its ranking changed significantly. It became competitive or best on larger `SafeTensors` and `ServerlessLLM` workloads. The final result is therefore

not "io_uring always wins", nor "io_uring is a dead end". The correct conclusion is that backend family and backend architecture both matter.

8.6 Summary

Three final findings emerge from the completed result set.

1. At the Rust layer, backend ranking depends strongly on workload structure. `SafeTensors` favours `sync` on smaller cases and `io_uring` on larger cases. `ServerlessLLM` favours `async` or `io_uring`, while `sync` is rarely best.
2. At the Python layer, binding overhead is large enough to matter, and binding-specific optimisation produced meaningful practical gains.
3. At the vLLM layer, backend choice affects initialisation and TTFT far more than steady-state decode, and the best loader depends on model size.

Taken together, the results support an adaptive, format-aware checkpoint-loading heuristic. The dominant practical rule is sync for smaller models and io_uring for larger ones, with additional format-specific adjustments for range-heavy layouts.

Chapter 9: Discussion and Critical Analysis

9.1 The Main Lesson: No Backend Wins Everywhere

The final result of this project is best understood as an adaptive-heuristic result. Backend performance depends on workload shape, checkpoint layout, and integration layer, which means checkpoint loading should be driven by workload-aware selection rather than by one fixed backend.

This is the most important correction to the early narrative. The project began with the hope that a single backend — ideally `io_uring` on Linux — might dominate across the board. The final evidence does not support that conclusion. At the Rust layer, `SafeTensors` and `ServerlessLLM` prefer different backends in different size regimes. At the Python layer, binding overhead changes the practical ordering. At the vLLM layer, model initialisation dominates while steady-state decode is largely insensitive to the loading backend.

That is why the final system is organised around an adaptive `default` policy. In the clearest high-level form, the heuristic is sync for smaller eager loads and io_uring for larger ones, with format-specific refinements where the access pattern changes.

9.2 Why the SafeTensors and ServerlessLLM Stories Diverge

The results diverge because the two formats induce different I/O structures.

SafeTensors is fundamentally a whole-file or multi-shard eager-loading problem. The dominant costs are reading large contiguous files and coordinating shard-level parallelism. In this regime, `sync` remains highly competitive for smaller models because its thread-parallel shard loading is simple and effective. As shard count and checkpoint size increase, the multi-worker `io_uring` backend becomes increasingly attractive because it can sustain more in-flight work without the same blocking behaviour.

ServerlessLLM is fundamentally a range-heavy partitioned-loading problem. The dominant costs are request grouping, range batching, and fragmented access across partitions. In this regime, the synchronous backend is rarely ideal. The competition is instead between a well-grouped async path and the multi-worker `io_uring` path. This is why the explicit `async` backend is strong for some mid-sized `ServerlessLLM` cases even when it is weak for `SafeTensors`.

The practical implication is clear: backend choice cannot be separated from layout choice. A loader that looks strong on `SafeTensors` does not automatically remain strong on a partitioned format, so any good heuristic has to look at workload structure rather than model size alone.

9.3 The Architectural Turnaround Matters More Than the Backend Label

One of the most useful lessons from the project is that backend family names can be misleading if the architecture behind them is poor.

The early single-ring `io_uring` backend performed badly enough that it would have been easy to conclude that `io_uring` was simply the wrong fit for checkpoint loading. That conclusion would have been wrong. Profiling showed that the prototype underutilised the machine and spent too much time in the single submission/completion path. The problem was not "Linux async I/O is bad for this workload"; the problem was "this specific `io_uring` architecture is bad for this workload".

After the redesign to multiple worker-owned rings, the `io_uring` backend changed from a negative result into a strong explicit contender. That shift is central to the interpretation of the work. It shows that backend evaluation for checkpoint loading must account not only for API family but also for execution topology, queue ownership, and batching structure.

This also explains why the report should preserve the turnaround story rather than pretending the earlier negative result never happened. The failed prototype was informative because it revealed which aspects of the architecture actually mattered.

9.4 Why Python Needed Its Own Optimisation Work

A naïve reading of the Rust results might suggest that the Python layer should simply inherit the same backend ordering. The experiments show that this is not true. Python introduces substantial extra work: runtime orchestration, tensor conversion, Torch dtype resolution, and dictionary materialisation. Those costs can swamp part of the raw I/O gain.

This is why the binding-specific optimisation pass was necessary. Reusing a shared Tokio runtime and reducing repeated Torch conversion overhead materially improved the public `default` path on Qwen3-8B. The important engineering lesson is that a fast core does not automatically imply a fast binding layer. If the goal is practical usability, the binding path must be profiled and optimised in its own right.

This does not weaken the backend conclusions. It refines them. The Rust layer answers which backend is best for raw checkpoint loading. The Python layer answers how much of that gain survives crossing the language boundary. The final system needs both answers.

9.5 What the vLLM Results Add

The vLLM results are valuable because they show how backend effects survive contact with a real serving stack.

The first lesson is that backend gains matter most for model initialisation and TTFT. That is where weight loading is still part of the critical path. The second lesson is that decode throughput is much less sensitive to backend choice, because once the model is resident, the GPU compute path dominates.

The third lesson is methodological. The initial failures of the 14B and 32B vLLM runs looked, at first glance, like large-model infeasibility. In fact, they were caused by conservative vLLM harness settings for memory budgeting and maximum model length. Once those settings were corrected, the full matrix completed successfully. This is a useful reminder that end-to-end integration experiments can fail for harness reasons that are orthogonal to the backend under study.

In practical terms, the vLLM matrix strengthens the project's central argument. A useful loading system is not judged only by backend microbenchmarks; it is judged by whether the selected backend helps the actual serving stack where it matters, especially during cold start.

9.6 What Matters for Practitioners

Three practitioner-oriented conclusions follow directly from the completed experiments.

First, prefer policy over dogma. Engineers should resist the temptation to standardise on one backend globally. The results do not justify such a rule. The right practical design is an adaptive selector with explicit override paths.

Second, optimise at the right layer. If the problem is Rust cold-load latency, backend architecture matters most. If the problem is Python user-facing load time, binding overhead can be equally important. If the problem is serving cold-start latency, vLLM setup and KV-cache configuration must also be considered.

Third, evaluate formats and loaders together. The best eager-loading backend is not a property of the backend alone. It emerges from the interaction between checkpoint layout and loader implementation.

9.7 Limitations

The report's conclusions should be interpreted with several limitations in mind.

Hardware specificity. All final measurements come from one H100-based Linux environment. The ordering of results is likely to generalise more robustly than the absolute timings, but further cross-hardware validation would strengthen that claim.

Heuristic imperfection. The final `default` selector is much better than the earlier cost-model approach, but it is still not perfect on every Rust matrix row. The medium-size crossover region remains the most difficult area to classify cleanly.

Serving-stack dependence. vLLM results are sensitive to engine settings such as memory budget and maximum model length. This is an unavoidable property of evaluating inside a real serving stack, but it means that the exact numbers are partly harness-dependent.

Scope. The report covers two checkpoint layouts and a six-model ladder. That is much stronger than the earlier prototype setup, but it still does not exhaust the design space of checkpoint formats or storage hardware.

9.8 Final Assessment

The project succeeds if judged against its final, more realistic goal. It does not identify one universally superior checkpoint-loading backend. Instead, it demonstrates that:

1. backend ranking depends on workload structure,
2. backend architecture matters as much as backend family,
3. Python and serving layers can materially change how backend gains appear in practice, and
4. an adaptive default policy is the most defensible design target.

That final conclusion is stronger than the original, simpler hypothesis would have been. It is also more useful. A practical checkpoint-loading system should not ask users to know which backend is best in every situation. It should use an adaptive heuristic that defaults to the simpler synchronous path when the checkpoint is small enough, moves to io_uring when larger checkpoints justify the extra machinery, and still leaves room for explicit overrides where experimentation or deployment constraints require them.

Chapter 10: Professional Issues

This chapter discusses the professional issues arising from the design, implementation, and evaluation of TensorStore, with particular attention to how each issue manifested during the project and what effect it had on technical decisions. Four topics are covered: reproducibility in systems benchmarking, correctness and safety in systems code, software supply chain management, and transparency in performance claims. These topics were selected because they are the areas where systems engineering decisions have the most direct professional consequences, and each one materially shaped how this project was conducted.

10.1 Reproducibility in Systems Benchmarking

Rigorous reproducibility is essential in systems performance research. A result that cannot be reproduced under the same conditions is not a reliable basis for engineering decisions or scientific claims. The Reproducibility Project found that the majority of published results in experimental computer science could not be reproduced with identical results, even with access to the original code and hardware [3]. While that study focused on machine learning, the same concerns apply to systems performance work.

During this project, a reproducibility issue arose that had a direct effect on the technical direction. Early benchmark runs produced results that appeared to show little performance difference between the synchronous and io_uring backends. This was puzzling given the characteristics of the storage stack, and it took systematic investigation to identify the cause: the page cache had retained checkpoint pages from earlier runs, so supposedly cold measurements were in fact loading from RAM rather than from disk. The misleading results persisted until the issue was identified through storage-level monitoring.

This experience directly changed how the methodology was designed. The final cold-cache procedure moved out of the code and into an explicit Linux-side benchmark ritual using `sync` and `drop_caches`. It also prompted the addition of explicit verification steps: cold-cache runs were interpreted together with storage-level evidence rather than accepted at face value. The lesson generalised beyond this project: benchmark methodology errors can persist for a long time before being identified, and system-level monitoring is necessary to validate that benchmarks measure what they claim to measure.

The project also addresses reproducibility through the fixed workload trace. The set of byte-range requests used in each experiment is derived directly from checkpoint metadata and is not altered between backends. This means that any researcher with access to the same checkpoint files and the same benchmark code can reproduce the exact same workload. The environment is described precisely, with hardware configuration, kernel version, filesystem type, and mount options all reported.

10.2 Correctness and Safety in Systems Code

Checkpoint loading code operates on untrusted data from disk. If a checkpoint file is corrupted or maliciously crafted, the loader must handle it gracefully without crashing or leaking memory. In production inference systems, checkpoint loaders run with the same privileges as

the inference server, which may itself be handling untrusted input. A buffer overflow in the loader could be exploited to gain code execution on the inference host.

This concern had a direct effect on the choice of implementation language. Rust was selected for the core backend specifically because its ownership model and borrow checker provide strong guarantees against memory-safety vulnerabilities at compile time. In C or C++, a mistake such as returning a borrowed reference to a dropped buffer would manifest as undefined behaviour at runtime, potentially corrupting benchmark results in subtle ways. In Rust, such an error is caught by the compiler. The SafeTensors format was chosen partly for the same reason: it prohibits unsafe deserialisation, unlike Python pickle, which can execute arbitrary code during load [5]. TensorStore implements the SafeTensors parser in Rust, adding the borrow checker on top of the format's own safety guarantees.

When binding Rust code to Python via PyO3, care is required at the language boundary. The PyO3 bindings use typed wrappers that validate Python objects at the boundary, preventing invalid values from propagating into Rust. GIL management ensures that Rust code does not interfere with Python's interpreter state during I/O operations. This was a deliberate design choice to maintain the safety guarantees of the Rust core through the Python binding layer.

Correctness testing combines unit tests at the format layer with integration tests at the binding layer. Unit tests verify that the format parser produces the correct byte-range requests from synthetic checkpoint files. Integration tests verify that each backend produces bit-identical tensor data by comparing against the HuggingFace reference implementation. These tests caught a regression during development where an off-by-one error in the byte offset calculation produced incorrect tensor shapes for multi-partition loads.

10.3 Software Supply Chain

The TensorStore project depends on external software components maintained by third parties. Managing these dependencies is a significant professional responsibility, because vulnerabilities in transitive dependencies become vulnerabilities in the project.

The supply chain considerations influenced several decisions during development. Dependency versions are pinned in `Cargo.lock` to ensure that builds are reproducible and that updates are reviewed explicitly rather than applied automatically. The project uses `cargo-audit` and strict linting in CI to catch dependency and code-quality issues early.

One supply chain issue that arose during development was drift between the Rust core and its surrounding tooling layers. The project eventually removed stale `tokio-uring` assumptions from the Python bindings and documentation so that the bindings matched the final low-level `io-uring` implementation in the Rust core. This incident reinforced the same professional lesson as version pinning: ecosystem drift that goes unreviewed can quietly invalidate experiments and documentation.

The HuggingFace Hub is used as a source of real checkpoint files for testing. The Hub is an external service, and the availability of checkpoints depends on its continued operation. The project caches downloaded checkpoints locally, which reduces but does not eliminate this dependence. For the evaluation runs, checkpoints were verified to be present before benchmarking began, and a local cache allowed rapid re-running of experiments when the Hub was temporarily unreachable.

The PyO3 and maturin build system introduces a dependency on the Python interpreter

version. Changes to Python's C API between versions can affect generated bindings. The project documents the supported Python version range and tests against multiple versions in CI, which mitigated this risk during development.

10.4 Transparency in Performance Claims

Performance claims in systems research carry professional responsibility. Overstating results or omitting inconvenient data undermines the reliability of the research record and can lead to poor engineering decisions based on misleading information. The BCS Code of Conduct for Professional Engineers emphasises honesty and accuracy in the presentation of findings [12].

This report has been transparent about both positive and negative results. The warm-cache measurement flaw is documented in the results chapter, and the removal of the async vLLM loader path is explained in terms of architectural constraints rather than hidden or minimised. The async vLLM path was an attempted integration that turned out to be architecturally unsound; the honest account of why it was removed is more professionally appropriate than presenting only the backends that worked.

The distinction between load-time improvement and end-to-end latency improvement is maintained throughout. Backend-level gains do not automatically translate to user-visible improvements; the vLLM integration results are reported separately from the Rust-level results precisely because the integration context introduces additional components that may dominate the overall cost. This separation reflects the professional obligation to present results in a way that accurately represents their scope.

The BCS Code of Conduct provides the frame for this standard: presenting results as measured, including measurements that do not support initial hypotheses, is not just good scientific practice but a professional obligation. The project diary maintained throughout development records when findings diverged from expectations, which provides an audit trail for the decision to document negative results rather than exclude them.

10.5 Summary

The professional issues arising from this project centre on the responsibility that comes with making performance claims, distributing open-source code, and operating in a complex software supply chain. Each of these areas had a demonstrable effect on the project. The reproducibility concern led directly to a methodological change that was essential for valid results. The correctness and safety concern influenced the choice of Rust as the implementation language and the SafeTensors format. The supply chain concern led to explicit dependency management practices and CI tooling. The transparency concern shaped how negative results are presented. The project aimed throughout to meet the standard implied by the BCS Code of Conduct: honest, accurate presentation of findings that reflects the actual state of the work.

Chapter 11:**Conclusion**

11.1 Research Questions Answered

This chapter returns directly to the research questions posed at the start of the project.

RQ1: How does backend choice affect checkpoint loading time for real LLM checkpoints?

Backend choice affects loading time substantially, but not uniformly. At the Rust layer, `SafeTensors` and `ServerlessLLM` produce different backend rankings, and those rankings also change with checkpoint size. Smaller `SafeTensors` loads favour `sync`, while larger `SafeTensors` loads favour the final multi-worker `io_uring` design. For `ServerlessLLM`, the strongest explicit backend is usually `async` or `io_uring`, with `sync` rarely best.

RQ2: How does checkpoint layout affect the relative performance of backends?

Checkpoint layout changes the effective workload. Whole-file shard loading and range-heavy partitioned loading stress different parts of the implementation. `SafeTensors` rewards strong bulk shard parallelism; `ServerlessLLM` rewards good request grouping, coalescing, and range scheduling. The evaluation therefore shows that layout and loader must be considered together rather than in isolation.

RQ3: How much overhead is introduced by the Python integration layer?

The Python layer introduces significant overhead through runtime orchestration, tensor conversion, and object materialisation. That overhead is large enough to distort backend-level gains if left untreated. However, binding-specific optimisation reduced the public `default` path substantially on Qwen3-8B, showing that this overhead can be meaningfully reduced through engineering work at the binding layer.

RQ4: To what extent do backend gains survive integration into vLLM?

Backend gains survive into vLLM most clearly for model initialisation and TTFT. They matter much less for steady-state decode, where GPU compute dominates. The completed vLLM matrix confirms that loader choice still matters in a serving stack, but it also confirms that backend microbenchmarks alone are not enough to predict end-to-end behaviour.

RQ5: What practical engineering lessons emerged?

The most important lesson is that the architecture of a backend matters as much as the backend family. The early single-ring `io_uring` prototype underperformed badly, but the final multi-worker design became competitive or best in several important regimes. The correct design target is therefore not one universal backend, but an adaptive, format-aware `default` policy backed by explicit override paths.

11.2 Main Contributions

The project makes four main contributions.

1. It builds a common Rust loading framework that allows direct comparison of `sync`, `async`, and `io_uring` under shared APIs and shared formats.
2. It demonstrates that the performance ranking of these backends depends on workload structure rather than on a single universal rule.
3. It shows that binding-layer and serving-layer overheads can materially alter how backend gains appear in practice.
4. It produces an adaptive default policy that is substantially more defensible than either a fixed backend choice or the earlier synthetic cost-model approach.

11.3 Limitations

The conclusions should still be interpreted with some care.

The experiments were run on one H100-based Linux environment, so the absolute timings are hardware-specific. The selector is much better than the earlier heuristic, but not perfect in every crossover case. The vLLM results are also partly conditioned on serving-engine settings such as memory budget and maximum model length. These limitations affect the exact quantitative boundaries between regimes, but they do not overturn the main qualitative result.

11.4 Final Conclusion

The final conclusion of the project is simple but stronger than the original hypothesis. The most useful way to think about LLM checkpoint loading is as an adaptive heuristic problem. The backend should not be fixed in advance; it should be selected from workload structure. In the clearest high-level form, the final evidence supports sync for smaller eager checkpoints and io_uring for larger ones, with format-specific refinements around that rule.

That is the central outcome of this work. The strongest practical policy is an adaptive default, informed by real workload structure and validated across Rust, Python, and vLLM layers.

Bibliography

[1] Jens Axboe. Efficient i/o with io_uring. `https://kernel.dk/io_uring.pdf`, 2019.

[2] Hao Cao and other authors. Guidelines for using io_uring in database systems, 2025.

[3] Steven W. D. Chouldechova et al. Reproducible research in computational science. *Nature*, 521:371–373, 2015. The Reproducibility Project.

[4] ServerlessLLM Contributors. Serverlessllm. `https://github.com/ServerlessLLM/ServerlessLLM`, 2024. GitHub repository.

[5] Hugging Face. Safetensors documentation. `https://huggingface.co/docs/safetensors`, 2024. Accessed: 2025.

[6] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. Serverlessllm: Low-latency serverless inference for large language models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*. USENIX Association, 2024.

[7] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 5th edition, 2012.

[8] Michael Kerrisk and contributors. *aio(7) Linux Manual Page*, 2024.

[9] Michael Kerrisk and contributors. *io_uring(7) Linux Manual Page*, 2024.

[10] NVM Express Organization. Nvm express specifications. `https://nvmexpress.org/specifications/`, 2024.

[11] Paulo Silva and Thiago Emmanuel Pereira. Performance evaluation of snapshot methods to warm the serverless cold start, 2021.

[12] British Computer Society. Code of conduct for professional engineers. `https://www.bcs.org/media/4411/code-of-conduct.pdf`, 2023. Accessed: 2025.

[13] vLLM Team. vllm: Easy, fast, and cheap llm serving with pagedattention. `https://github.com/vllm-project/vllm`, 2024.

Appendix A:**Backend Interface Specifications**

A.1 Rust Backend API

The core Rust backend interface for synchronous loading is defined in the Python bindings at `safetensors.rs` and `serverlessllm.rs`, within the `bindings/python/rust_ext/src/api/` package directory.

A.1.1 SafeTensors Synchronous Loading

The synchronous loading function validates the path, loads the checkpoint using the Rust `Model::load_sync` call within `py.allow_threads` to release the GIL, and converts each tensor to the target framework format.

A.1.2 ServerlessLLM Synchronous Loading

The synchronous loading function checks the partition count from the index file and dispatches to either `Model::load_parallel_sync` for multi-partition checkpoints or `Model::load_sync` for single-partition checkpoints.

A.2 Python Binding Signatures

A.2.1 load_safetensors_sync

```
load_safetensors_sync(
    path: str,
    framework: str = "torch",
    device: str = "cpu"
) -> dict
```

Loads a SafeTensors checkpoint synchronously. Returns a dictionary mapping tensor names to framework tensor objects.

A.2.2 load_safetensors_mmap

```
load_safetensors_mmap(
    path: str,
    framework: str = "torch",
    device: str = "cpu"
) -> dict
```

Opens a SafeTensors checkpoint as a memory map. Tensor access triggers on-demand page

loading.

A.2.3 load_serverlessllm_sync

```
load_serverlessllm_sync(
    path: str,
    framework: str = "torch",
    device: str = "cpu"
) -> dict
```

Loads a ServerlessLLM-style checkpoint synchronously. Automatically selects parallel or sequential loading based on partition count.

A.3 vLLM Loader Configuration

The TensorStore loader is registered under the name `tensor_store` and configured via `model_loader_extra_config`:

```
{
    "format": "safetensors" | "serverlessllm",
    "backend": "sync" | "mmap"
}
```

Appendix B:**Supplementary Results**

B.1 Additional Throughput Data

This appendix contains additional throughput measurements across different model sizes and checkpoint configurations. Results will be populated following the final experimental runs.

B.2 Latency Distributions

Figure B.1 shows the per-read latency distribution for the synchronous and io_uring backends under the SafeTensors layout. The CDF is constructed from all individual read operations recorded during the cold-cache throughput experiments.

[Figure to be populated with latency CDF data from experiment runs]

Synchronous vs io_uring per-read latency CDF, SafeTensors layout, cold-cache

Figure B.1: Cumulative distribution of per-read latency for synchronous and io_uring backends.

B.3 vLLM Integration Additional Data

Table B.1 shows steady-state decode latency measurements across loader configurations and cache conditions.

Table B.1: Steady-state decode latency (ms per token) by loader and cache condition.

Loader	Cache	Decode latency
native	warm	TBD
native	cold	TBD
ts_safetensors_sync	warm	TBD
ts_safetensors_sync	cold	TBD
ts_safetensors_mmap	warm	TBD
ts_safetensors_mmap	cold	TBD

B.4 Outstanding Experiment Outputs

The following outputs are required to complete this appendix:

- Per-read latency CDF data for synchronous and io_uring backends (Figure B.1)
- Steady-state decode latency for all six loader and cache configurations (Table B.1)